CPSC 368: Databases in Data Science

MongoDB

Learning Goals

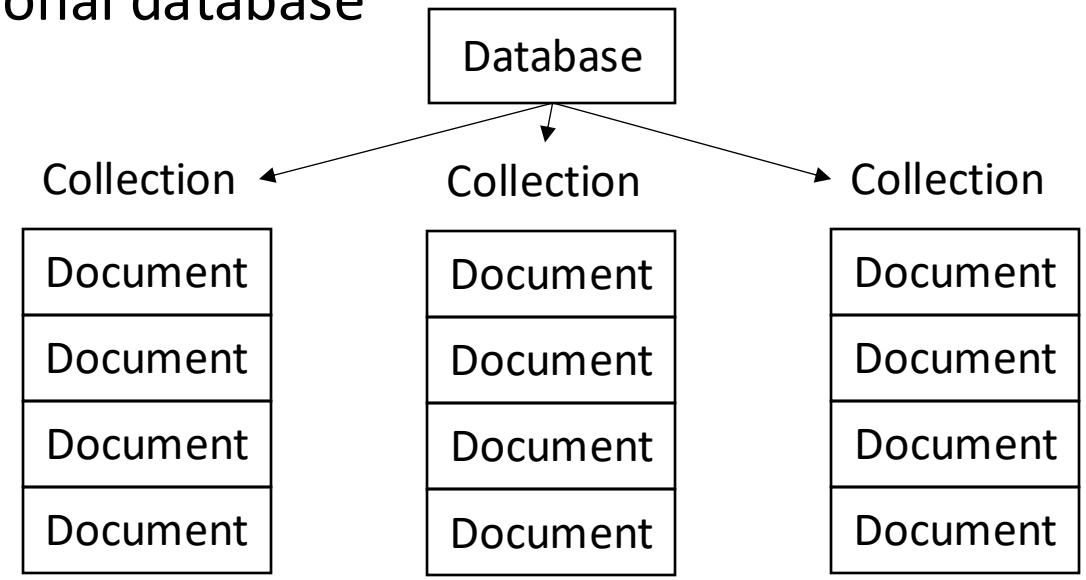
- Describe how MongoDB stores data
- Identify various parts of a JSON object
- Translate data from an ER diagram to a JSON object representation
- Translate between SQL and MQL
- Use Python to manipulate data in a MongoDB database

Let’s talk data storage…

- Relational databases use tables
- MongoDB uses JSON

What is MongoDB?

- MongoDB is a type of document database
  - Instead of tables, information is stored as JSON objects
  - Each JSON object is called a **document**
- A group of documents is called a **collection**
  - A collection is similar to a table in a relational database
- The system will assign an `_id` value to each document
- A document has a maximum size of 16MB



What is JSON

- JSON stands for JavaScript Object Notation
- Stores data as pairs of field name/value(s)

```
{
        "_id": 507f1f77bcf86cd799439011,
        "firstName": "Perry",
        "lastName": "Platypus",
        "jobs": ["family pet", "secret agent"],
        "address": {
                "buildingNo": 2366,
                "street": "Main Mall",
                "city": "Vancouver"
                "province": "BC"
        }
}
```

MongoDB and JSON

- Default behavior is to allow documents in a collection to have different schemas
- Schema validation allows you to impose rules on each document
- Can specify things like which attributes are absolutely required for each document, impose restrictions on values, etc.

What is JSON

```
{
        "_id": 507f1f77bcf86cd799439011,
        "firstName": "Perry",
        "lastName": "Platypus",
        "jobs": ["family pet", "secret agent"],
        "address": {
                "buildingNo": 2366,
                "street": "Main Mall",
                "city": "Vancouver"
                "province": "BC"
        }
}
```

What is JSON

```
{
        "_id": 507f1f77bcf86cd799439011,
        "firstName": "Perry",
        "lastName": "Platypus",
        "jobs": ["family pet", "secret agent"],
        "address": {
                "buildingNo": 2366,
                "street": "Main Mall",
                "city": "Vancouver"
                "province": "BC"
        }
}
```

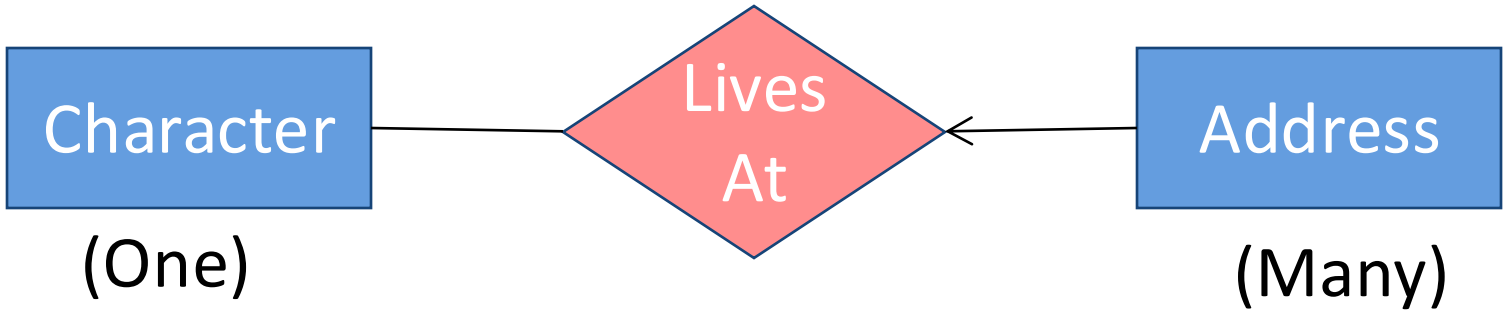
What is JSON

```
{
        "_id": 507f1f77bcf86cd799439011,
        "firstName": "Perry",
        "lastName": "Platypus",
        "jobs": ["family pet", "secret agent"],
        "address": {
                "buildingNo": 2366,
                "street": "Main Mall",
                "city": "Vancouver"
                "province": "BC"
        }
}
```

What is JSON

```
{
        "_id": 507f1f77bcf86cd799439011,
        "firstName": "Perry",
        "lastName": "Platypus",
        "jobs": ["family pet", "secret agent"],
        "address": {
                "buildingNo": 2366,
                "street": "Main Mall",
                "city": "Vancouver"
                "province": "BC"
        }
}
```

What is JSON




```
{
        "_id":  507f1f77bcf86cd799439011,
        "firstName": "Perry",
        "lastName": "Platypus",
        "jobs": ["family pet", "secret agent"],
        "address": [
                { "buildingNo": 2366, "street": "Main Mall", "city": "Vancouver",
                "province": "BC"},
                { "buildingNo": 6245, "street": "Agronomy Rd", "city": "Vancouver",
                "province": "BC"}
        ]
}
```

Embedded Documents

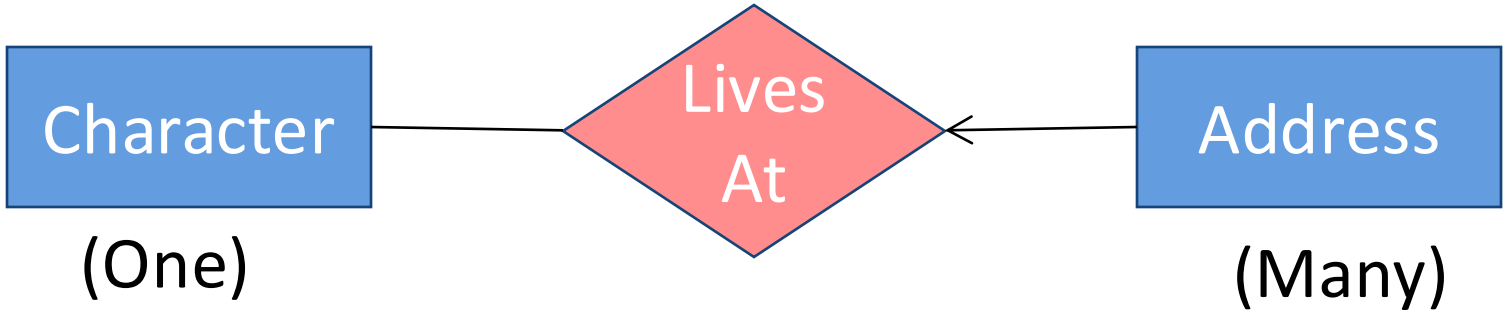
- A document has 0 to n embedded documents where n is a positive integer
- It helps keep data together which results in fewer queries and fewer update statements
  - Can read in all the relevant data at once
- Benefit of allowing updates to happen in one operation
- Possible issue with duplication of data
- You can nest up to 100 levels before it is unsupported

Referencing another document (Manual Reference)

```
{ “_id”:  123,  “buildingNo”: 2366, “street”: “Main Mall”, “city”: “Vancouver”,
”province”: “BC”},

{ ”_id”: 234, “buildingNo”: 6245, “street”: “Agronomy Rd”, “city”: “Vancouver”,
”province”: “BC”}
```

```
{“_id”: 345, “firstName”: “Perry”, ”lastName”: “Platypus”, “address”: ObjectId(”123")}
```



Referencing another document (DBRefs)

```
{ “_id”:  123,  “buildingNo”: 2366, “street”: “Main Mall”, “city”: “Vancouver”, ”province”: “BC”},

{ ”_id”: 234, “buildingNo”: 6245, “street”: “Agronomy Rd”, “city”: “Vancouver”, ”province”: “BC”}
```

```
{
        “_id”: 345,
        // other fields
        “address”: {
                "$ref" : ”address", // collection name
                "$id" : ObjectId(”123"), // id of the particular document
                "$db" : "users” // database name
        }
}
```

Referencing another document

- DBRefs method requires a particular attribute order
- Depending on which driver you are using, the behaviour may differ slightly
- You may want to consider using this method when referencing documents from multiple collections

Terminology Comparison

Relational Database	MongoDB
Database	Database
Table/View	Collection
Row	Document
Column	Field
Join	Embedded Document
Foreign Key	Reference

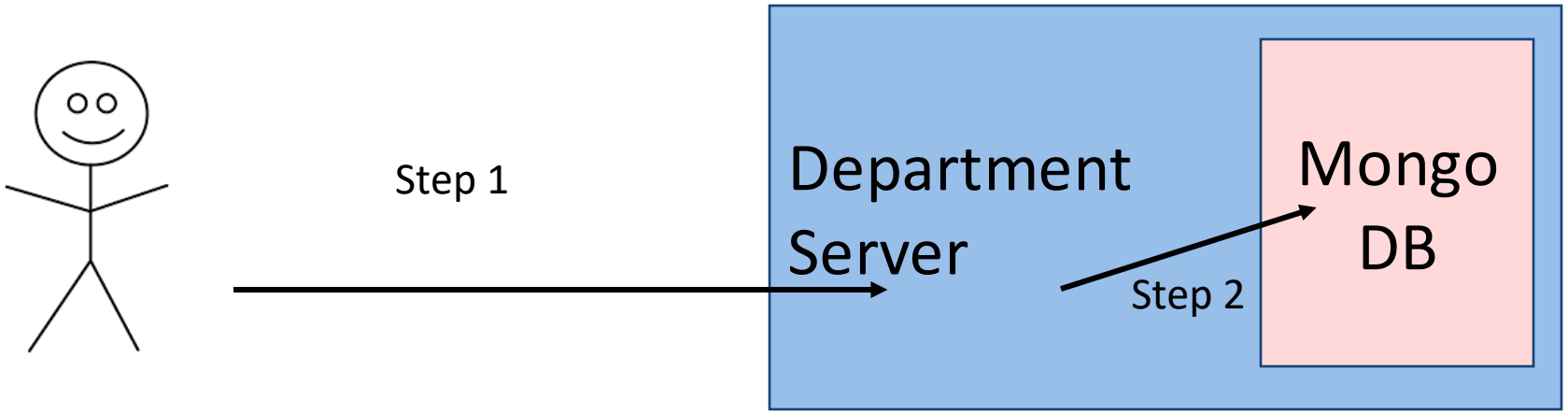
Let’s talk about querying data…

- CRUD
  - **C**reate
  - **R**ead
  - **U**pdate
  - **D**elete

- These are the four basic operations for any database
- Queries themselves resemble JSON objects
- You will get a chance to try this out in one of the assignments

Accessing the department's MongoDB installation

- The MongoDB server, much like the Oracle server, runs on a protected department host
- The computer that you use to access MongoDB needs to tunnel into the department before it will be allowed to log into the database
  - If you use Syzygy, tunnel from Syzygy
  - If you use your own computer, tunnel from your own computer



Create

- In SQL
  - `CREATE TABLE`
  - `INSERT`

- In MongoDB, inserting into a nonexistent collection will create it

```
db.inventory.insert_one(
    {item: "canvas", qty: 100, tags: ["cotton"],
size: { h: 28, w: 35.5, uom: "cm" } }
)
```

Create

* Assume db has been set to refer to the inventory collection

- Just like in a relational database, all documents must have a unique identifier
- If no `_id` attribute is provided, one will be automatically assigned upon insertion

```
db.insert_one(
 {“item”: "canvas", “qty”: 100, “tags”:
["cotton"], “size”: { “h”: 28, “w”: 35.5, “uom”:
"cm" }}
)
 db.insert_one(
    {“_id”:10, “item”: "canvas", “qty”: 100,
 “tags”: ["cotton"], "size”: { “h”: 28, “w”: 35.5,
 “uom”: "cm" } }
 )
```

Create

```
* Assume db has been set
to refer to the inventory
collection
```

- Lots of data to insert?
- Oracle lets you insert tuples returned from a query (other SQL databases like MySQL allow for batch insert)
- There is an equivalent method for MongoDB

```
db.insert_many(
   {“item”: "canvas", “qty”: 100, “tags”:
["cotton"], “size”: { “h”: 28, “w”: 35.5, “uom”:
"cm" } },
     {“item”: ”flannel", “qty”: 100, “tags”:
["cotton"], “size”: { “h”: 22, “w”: 27, “uom”:
"cm" } }
)
```

Read

- In SQL, we have queries like `SELECT _____ FROM _____`
- `db.collection_name.find({})`
  - Equivalent to `SELECT * FROM collection_name`
- WHERE clause can be replicated with search conditions inside the `{}`

```
db.collection_name.find({<field>:<value>})
```

- `SELECT * FROM Inventory WHERE name = “notebook”`

```
db.find({”name": ”notebook"})
```

* Assume db has been set to refer to the inventory collection

Read

- List multiple field/values to replicate AND

```
db.collection_name.find({<field>:<value>,
({<field>:<value>})
```

- SELECT * FROM Inventory WHERE name="notebook" and qty > 20

```
db.find({"name": "notebook", "qty": {"$gt":
30}})
```

* Assume db has been set to refer to the inventory collection

Read

* Assume db has been set to refer to the inventory collection

- You can use `$in` or `$or` to replicate `OR` behaviour

- `SELECT * FROM Inventory WHERE name="notebook" OR name="paper"`

  ```
  db.find({"name": {"$in": "notebook", "paper"}})
  ```

- `SELECT * FROM Inventory WHERE name="notebook" OR qty > 20`

  ```
  db.find({"$or": [{"name": "notebook"}, {"qty": {"$gt": 20}}]})
  ```

Read

```
* Assume db has been set
to refer to the inventory
collection
```

Find functions return a **cursor** which you can use to iterate over the results. For example:

```
cur = db.find({”name”: {"$in": “notebook”,
“paper”}})

for c in cur:
    print(c)
```

Read

- Lots of other query operators available (not just >, <, =)
- There is functionality similar to the LIKE operator in SQL
- https://docs.mongodb.com/manual/reference/operator/query/#std-label-query-selectors

Projection

- `SELECT _id, attr1, attr2 FROM tableName WHERE attr3 = value`

- `db.collection_name.find ({“attr3”: value}, {”attr1": 1, ”attr2": 1})`

- By default, `_id` is included unless its value is set to 0

- `db.collection_name.find ({“attr3”: value}, {”attr1": 1, ”attr2": 1, “_id”: 0})`

Update

- Two versions of the update method:
  - `update_one()`: Updates the **first** document that matches your condition
  - `update_many()`: Updates many documents at once
- Other possible methods to update a value include methods that can find and replace a document
- Can add a field into a document
- `UPDATE tableName SET attrName = value WHERE condition`
-
```
db.collection_name.update_one(
    {condition},
    {"$set": {”attrName": value}
)
```

Update

* Assume db has been set to refer to the inventory collection

- UPDATE Inventory SET qty = 5 WHERE name = “notebook”

- db.update_one(
  {“name”: “notebook”},
  {"$set": {”qty": 5}}
)

- You can also use the various update methods to add a new field into a document

Delete

- Two versions:
  - `delete_one()`: It will delete the **first** document it finds that matches
  - `delete_many()`: It will delete all the documents that match the given condition
- `DELETE FROM tableName WHERE condition`
- `db.collection_name.delete_many({condition})`

- `DELETE FROM tableName`
- `db.collection_name.delete_many({}) # danger!!!`

Aggregations

- Aggregations allow you to create multiple stages to process data step by step
- It is optimized for large datasets
- Documents can be modified dynamically
- Allows for analytics as it can support grouping, aggregate operators (e.g., count, sum) over data, and statistical calculations.

Common Aggregation Stages

Stage	Description
$match	Filters documents (like SQL's WHERE)
$group	Groups documents by a specific field (like SQL's GROUP BY)
$sort	Sorts documents (like SQL's ORDER BY)
$project	Reshapes the document (includes/excludes fields)
$limit	Limits the number of documents
$skip	Skips a specified number of documents
$count	Counts the number of documents
$unwind	Deconstructs an array field into multiple documents
$lookup	Performs a left outer join with another collection

Example Collection: Order v1

- Orders

{ "_id": 1, "customer": "Alice", "amount": 50, "category": "Books" }
{ "_id": 2, "customer": "Bob", "amount": 100, "category": "Electronics" }
{ "_id": 3, "customer": "Alice", "amount": 30, "category": "Books" }
{ "_id": 4, "customer": "Alice", "amount": 20, "category": "Electronics" }

Grouping and Aggregates

```
SELECT customer, SUM(amount)
FROM Orders
GROUP BY customer
```

```
db.orders.aggregate([
  { $group: { _id: "$customer", totalSpent: { $sum: "$amount" } } }
])
```

```
[
  { "_id": "Alice", "totalSpent": 100 },
  { "_id": "Bob", "totalSpent": 100 }
]
```

Filtering and Ordering

```
SELECT customer, SUM(amount)
FROM Orders
WHERE category = “Books”
GROUP BY customer
ORDER BY SUM(amount) DESC
```

```
[
  { "_id": "Alice", "totalSpent": 80 }
]
```

```
db.orders.aggregate([
  { $match: { category: "Books" } },
  { $group: { _id: "$customer", totalSpent: { $sum: "$amount" } } },
  { $sort: { totalSpent: -1 } }
])
```

Example Collection: Order v2

- Orders

{ "_id": 1, "customerId": 101, "amount": 50 }
{ "_id": 2, "customerId": 102, "amount": 100 }

- Customers

{ "_id": 101, "name": "Alice" }
{ "_id": 102, "name": "Bob" }

Lookup

```
SELECT o.id, o.customerID, o.amount, c.id, c.name
FROM Orders o, Customer c
WHERE o.customerID = c.id

db.orders.aggregate([
  {
    $lookup: {
      from: "customers",
      localField: "customerId",
      foreignField: "_id",
      as: "customerDetails"
    }
  }
])
```

```
[
  {
    "_id": 1,
    "customerId": 101,
    "amount": 50,
    "customerDetails": [ { "_id": 101, "name": "Alice" } ]
  },
  {
    "_id": 2,
    "customerId": 102,
    "amount": 100,
    "customerDetails": [ { "_id": 102, "name": "Bob" } ]
  }
]
```

Useful Links

- SQL to MongoDB mapping
- SQL to aggregation mapping chart
- Aggregate operators
- Aggregation stages